CENTRO UNIVERSITÁRIO FACIMP WYDEN — UNIFACIMP
CURSO DE BACHARELADO EM SISTEMAS DE INFORMAÇÃO
DISCIPLINA: DESENVOLVIMENTO WEB

GUILHERME CORREIA

AUTORENT: SISTEMA WEB PARA GESTÃO DE FROTA E CONTROLE DE LOCAÇÃO DE VEÍCULOS

Etapa 1 — Definição do Problema, Requisitos Iniciais e Planejamento Técnico

Documento de concepção e planejamento técnico apresentado à disciplina de Desenvolvimento Web, do Curso de Sistemas de Informação do Centro Universitário FACIMP Wyden, como requisito avaliativo da Etapa 1 do Projeto Prático.

Professor: Esp. Welliton Cunha

Quadro 1 — Identificação do Projeto e Repositório

Nome provisório do projeto	AutoRent — Gestão de Frota e Locação de Veículos
Acadêmico responsável	Guilherme Correia
Curso e Turma	Sistemas de Informação — 2026.2 (Quarta-feira)
Repositório Git (GitHub)	https://github.com/gcorreia-devsec/autorent-web

IMPERATRIZ — MA
2026

1 PROBLEMÁTICA E CONTEXTO ATUAL

O projeto tem como nome provisório **AutoRent** e surgiu a partir de uma dificuldade bastante comum em pequenas e médias locadoras de veículos da região: a falta de um controle centralizado sobre a situação da frota e os prazos de devolução. Quando a empresa não tem clareza rápida sobre quais carros estão livres no pátio, quais estão alugados e quais pararam na oficina para manutenção, o atendimento fica lento e sujeito a falhas.

No contexto atual, muitas dessas locadoras menores ainda fazem esse controle de forma manual, usando planilhas soltas no Excel, anotações em cadernos ou trocando mensagens no WhatsApp. Na prática, isso gera vários problemas no dia a dia: um atendente pode reservar um veículo que acabou de sair ou que precisava de revisão mecânica, clientes perdem tempo no balcão esperando a conferência manual de disponibilidade e o administrador não consegue consultar um histórico confiável de quem alugou cada carro e quando ele deve ser entregue.

2 PÚBLICO-ALVO E PRINCIPAIS PESSOAS USUÁRIAS

O sistema é voltado para o uso interno de locadoras de veículos de pequeno e médio porte que precisam organizar sua operação diária de forma prática. As principais pessoas usuárias que vão interagir diretamente com o sistema são:

- **Atendentes de balcão:** são os operadores que mais utilizam o sistema na rotina diária, cadastrando clientes, consultando rapidamente quais veículos estão disponíveis no pátio e registrando a saída e a devolução de cada locação.
- **Gerentes ou proprietários da locadora:** utilizam o sistema para acompanhar a taxa de ocupação da frota, cadastrar novos veículos adquiridos, atualizar valores de diárias e consultar o histórico geral de contratos.
- **Responsáveis pelo pátio e manutenção:** consultam a listagem da frota para atualizar o status dos veículos que precisam de revisão mecânica ou lavagem antes de voltarem a ficar disponíveis para novos clientes.

3 OBJETIVO DA SOLUÇÃO E ESCOPO INICIAL

O objetivo geral do **AutoRent** é desenvolver uma aplicação web simples, direta e funcional (integrando frontend, API REST no backend e banco de dados relacional) que centralize o cadastro da frota, dos clientes e o registro das locações. Com isso, espera-se eliminar o risco de reservas duplicadas para o mesmo carro, diminuir o tempo de atendimento no balcão e manter o status de cada veículo atualizado em tempo real.

- **Dentro do escopo inicial:** cadastro, listagem, edição e exclusão (CRUD) de veículos (modelo, placa, marca, ano, valor da diária e status); CRUD de clientes (nome, CPF, CNH e telefone); registro de locação vinculando cliente e veículo com datas de saída e entrega; mudança automática do status entre *Disponível*, *Alugado* e *Manutenção*; e painel inicial com filtro da frota.
- **Fora do escopo inicial:** integração com meios de pagamento online (PIX ou cartão de crédito); emissão de Nota Fiscal Eletrônica (NF-e); rastreamento veicular por GPS em tempo real; e consulta automática de multas de trânsito.

4 REQUISITOS INICIAIS DO SISTEMA

A partir do problema identificado e do escopo inicial definido para o projeto, foram levantados os requisitos funcionais (o que o sistema deverá executar) e os requisitos não funcionais (critérios técnicos de arquitetura, segurança e usabilidade).

4.1 Requisitos Funcionais (RF)

Quadro 2 — Levantamento dos Requisitos Funcionais Iniciais

Código	Requisito	Descrição do Comportamento no Sistema
RF01	Gerenciar Veículos	Permitir cadastrar, listar, buscar por ID, editar e excluir veículos da frota, informando placa, modelo, marca, ano, valor da diária e status atual.
RF02	Gerenciar Clientes	Permitir cadastrar, consultar, atualizar e remover clientes da locadora, registrando nome completo, CPF, número da CNH, telefone e e-mail.
RF03	Registrar Locação	Permitir abrir uma nova locação vinculando um cliente cadastrado a um veículo com status "Disponível", informando a data de retirada e a previsão de devolução.
RF04	Atualizar Status	Alterar automaticamente o status do veículo para "Alugado" ao confirmar uma locação, impedindo que o carro seja selecionado em outro contrato simultâneo.
RF05	Registrar Devolução	Permitir encerrar uma locação ativa, calculando o valor total conforme as diárias utilizadas e retornando o status do veículo para "Disponível".
RF06	Filtrar Frota	Permitir filtrar rapidamente a listagem de veículos na tela inicial pelo status atual (Disponível, Alugado ou em Manutenção).

Fonte: Elaborado pelo autor (2026).

4.2 Requisitos Não Funcionais (RNF)

- **RNF01 — Arquitetura da API e separação de camadas:** o backend deve funcionar como uma API REST comunicando-se em formato JSON e mantendo o código organizado nas pastas `routes` (rotas), `controllers` (regras e status HTTP) e `models` (queries SQL).
- **RNF02 — Persistência e integridade relacional:** os dados devem ser persistidos no banco de dados PostgreSQL, utilizando restrições `UNIQUE` para placa, CPF e CNH, além de chaves estrangeiras (*Foreign Keys*) entre clientes, veículos e locações.
- **RNF03 — Segurança de credenciais e consultas:** variáveis sensíveis de banco (host, porta, usuário e senha) devem ficar isoladas em arquivo `.env` ignorado pelo Git, e todas as consultas SQL devem usar parâmetros posicionais (`$1, $2`) contra *SQL Injection*.
- **RNF04 — Usabilidade:** a interface web deve ser limpa, responsiva e validar campos obrigatórios nos formulários antes de enviar requisições para a API.

5 TECNOLOGIAS ESCOLHIDAS E LISTA DE FERRAMENTAS

As tecnologias foram escolhidas de acordo com o conteúdo prático da disciplina de Desenvolvimento Web, utilizando JavaScript tanto no frontend quanto no backend para manter o desenvolvimento individual padronizado e produtivo.

Quadro 3 — Lista de Tecnologias e Justificativa Técnica

Camada / Categoria	Tecnologia Escolhida	Justificativa e Papel no Projeto
Frontend	HTML5, CSS3 e JavaScript (ES6+)	Construção de telas rápidas e diretas para o balcão de atendimento, consumindo as rotas da API via `fetch()`.
Ambiente Backend	Node.js + npm	Execução de JavaScript no lado do servidor com suporte assíncrono e gerenciamento de bibliotecas através do `npm`.
Framework Web	Express.js	Estruturação ágil do servidor HTTP, organização de rotas (`GET`, `POST`, `PUT`, `DELETE`) e uso do middleware `express.json()`.
Banco de Dados	PostgreSQL + pgAdmin 4	SGBD relacional gratuito e seguro para armazenar as tabelas de veículos, clientes e contratos com integridade referencial.
Integração e Ambiente	Pacotes `pg`, `dotenv`, `cors` e `nodemon`	O `pg` gerencia o *Pool* de conexões com o PostgreSQL; o `dotenv` protege as senhas no `.env`; o `cors` libera o acesso do frontend e o `nodemon` reinicia o servidor automaticamente.
Prototipação	Figma	Desenho e validação das telas e fluxos principais do sistema antes da codificação completa da interface.
Versionamento e Testes	Git, GitHub, VS Code e Postman	Git e GitHub para controle de versão do repositório, VS Code para codificação e Postman para testar os endpoints da API.

Fonte: Elaborado pelo autor (2026).

6 RISCOS, DÚVIDAS E PREMISSAS IDENTIFICADOS

- **Riscos identificados:** possibilidade de sobrecarga de tarefas ou atraso na integração entre o frontend e a API por se tratar de um projeto desenvolvido individualmente, além do cuidado necessário na lógica para impedir reservas de um mesmo veículo em datas conflitantes.
- **Dúvidas iniciais:** confirmar com o professor se a demonstração final do sistema poderá ser realizada diretamente em ambiente local (`localhost`) no computador de desenvolvimento ou se será exigido o *deploy* em nuvem.
- **Premissas:** parte-se do princípio de que o ambiente de desenvolvimento local já conta com todas as ferramentas instaladas e configuradas (Node.js, PostgreSQL, pgAdmin, VS Code, Git e Postman) para o andamento semanal do projeto.